



Módulo 4

Enlace a Datos

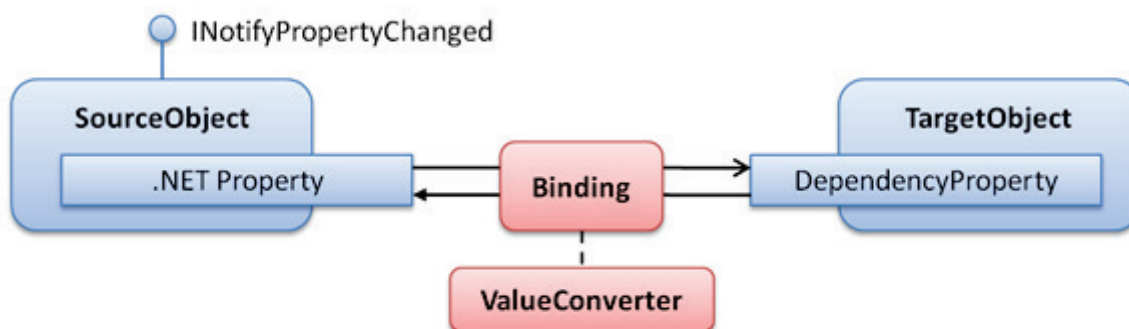
1- Enlazar Datos

Enlace a Datos

El enlazado de datos es el proceso que permite establecer una conexión entre la interfase de usuario y la lógica de negocios. Si el enlace es correcto y los datos proveen notificaciones correctamente, al cambiar los datos los elementos enlazados a ellos reflejan el cambio automáticamente y a su vez también debería funcionar en sentido inverso, o sea si modificamos el valor sobre los elementos, el dato asociado también se modifica.

WPF proporciona una forma eficaz de enlace de datos. Con WPF, puede realizar la manipulación de datos mediante código Microsoft® .NET Framework, XAML o una combinación de ambos. Puede realizar el enlace a controles, propiedades públicas, XML u objetos, convirtiendo las operaciones de enlace de datos en tareas rápidas, flexibles y realmente fáciles. Para usar el enlace de datos de WPF, debe disponer siempre de un destino y un origen.

- El destino del enlace puede ser cualquier elemento o propiedad accesible que se derive de *DependencyProperty*, un ejemplo es la propiedad *Text* del control *TextBox*.
- El origen del enlace puede ser cualquier propiedad Public, incluidas propiedades de otros controles, objetos de common language runtime (CLR), elementos XAML, *DataSets* de ADO.NET, fragmentos XML, etc.



Clase Binding

Define un enlace que conecta las propiedades de destinos de enlace y de orígenes de datos. Cada enlace debe especificar el elemento de destino, la propiedad de destino y el origen de datos. Las propiedades de uso más común de esta clase son:

- Source: Referencia a la fuente de datos.
- ElementName: El nombre del control al cual enlazamos una propiedad, se usa como alternativa de Source cuando el enlace es entre controles.
- Path: Indica la propiedad del elemento al cual enlazaremos el dato
- Converter: Instancia de una clase que implementa la interfase *IValueConverter*, que intercepta los movimientos de datos entre el origen y el destino de los datos o viceversa y permite la conversión de datos, el formateo de los mismo, etc. Son muy convenientes y se utilizan con frecuencia

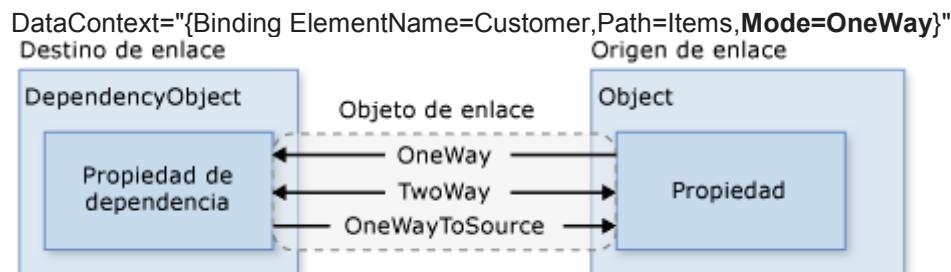
Al crear un enlace se usará la propiedad *ElementName* cuando el enlace se genera contra otro elemento WPF. Se usará la propiedad *Source* cuando el objeto a enlazar no es un elemento WPF, por ejemplo cuando es un recurso. No es necesario especificar ninguno de esas dos propiedades cuando el enlace se hace mediando el uso del objeto *DataContext*.

Modos de Enlazar – Propiedad Mode

La etiqueta *Binding* permite también el atributo *Mode*. El atributo *Mode* define el modo de enlace que determinará el flujo de datos entre el origen y el destino.

- OneWay: los datos fluyen desde el origen hasta el destino cada vez que se realiza un cambio en el origen. Es el modo predeterminado
- OneTime: Envía datos desde el origen al destino; sin embargo, hace esto sólo cuando se inicia la aplicación o cuando cambia el DataContext y, como resultado, no espera notificaciones de cambio en el origen.
- OneWayToSource: Envía datos desde el destino al origen.
- TwoWay: Envía los datos del origen al destino y si hay cambios en el valor de la propiedad de destino, éstos volverán a enviarse al origen.

La selección del modo de enlace apropiado es muy importante. Con frecuencia se emplea *OneWay* cuando se desea mostrar los datos a un usuario. Se usa *TwoWay* cuando se desea que el usuario pueda cambiar los datos en el control y que ese cambio se refleje en el origen de datos (por ejemplo un DataSet, objeto, XML u otro control de enlace). Se usa *OneWayToSource* cuando se desea permitir a un usuario cambiar el origen de datos sin que el origen de datos tenga que volver a enlazar sus datos al destino y se usa *OneTime* cuando se desea cargar datos de solo lectura al cargar la pantalla y que estos permanezcan sin cambios aunque se efectúen cambios en el origen de datos.



Momento del Enlace

Por defecto el momento del enlace esta predefinido en cada control. Por ejemplo en el caso del *ListBox* el enlace se hace al momento de seleccionar un elemento, en cambio en el caso de un *TextBox* el enlace ocurre al perder el foco. Para cambiar el evento que provoca la devolución de los datos al origen, puede especificar un valor para el atributo **UpdateSourceTrigger**, que es la propiedad de enlace que define cuándo debe actualizarse el origen. Hay cuatro valores que pueden configurarse para *UpdateSourceTrigger*:

- Default: se actualizará en el evento asociado por defecto al control
- Explicit: el origen no se actualizará a menos que se llame desde el código al método *UpdateSource* de la clase *BindingExpression*.
- LostFocus: indica que el origen se actualizará cuando el control del destino pierda el foco.
- PropertyChanged: indica que el destino actualizará el origen cada vez que cambie la propiedad enlazada del control del destino. Esta configuración es útil si desea establecer el momento del enlace de forma personalizada.

En este ejemplo a medida que escribe en el *TextBox* se va escribiendo en el *TextBlock*, ya que el enlazado está sobre la propiedad *Text* y el momento establecido con la opción **PropertyChanged** es la modificación de esa propiedad:

```
<StackPanel>
  <StackPanel.Resources>
    <TextBlock x:Key=" miFuente "></TextBlock>
  </StackPanel.Resources>
```



```
<Label>Ingrese el nombre:</Label>
<TextBox>
    <TextBox.Text>
        <Binding Source="{StaticResource miFuente }" Path="Name"
        UpdateSourceTrigger="PropertyChanged"/>
    </TextBox.Text>
</TextBox>
<Label>El nombre ingresado es:</Label>
<TextBlock Text="{Binding Source={StaticResource miFuente }, Path=Name}"/>
</StackPanel>
```

Si modifica el valor de *UpdateSourceTrigger* a **Default**, el *TextBlock* se actualizará al perder el foco del *TextBox*, ya que ese es el evento predeterminado para ese control. En cambio si establece esta propiedad en **Explicit** deberá invocar al método *UpdateSource* de la clase *BindingExpression*.

En el siguiente ejemplo se agregó un botón que en su evento *Click* que invoca a dicho método. De esta manera para actualizar el dato deberá presionar el botón que crea una instancia de la clase **BindingExpression** e invoca al método **UpdateSource**.

```
<StackPanel>
    <StackPanel.Resources>
        <TextBlock x:Key="miFuente"></TextBlock>
    </StackPanel.Resources>
    <Label>Ingrese el Nombre:</Label>
    <TextBox Name="txtNombre">
        <TextBox.Text>
            <Binding Source="{StaticResource miFuente}" Path="Name"
            UpdateSourceTrigger="Explicit" />
        </TextBox.Text>
    </TextBox>
    <Button Height="23" Width="70" Content="Actualizar" Click="Button_Click"></Button>
    <Label>El nombre ingresado:</Label>
    <TextBlock Text="{Binding Source={StaticResource miFuente}, Path=Name}"/>
</StackPanel>
```

En la ventana de código:

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    BindingExpression be =
        txtNombre.GetBindingExpression(TextBox.TextProperty);
    be.UpdateSource();
}
```

Propiedad Asociadas al Enlace de Datos

BindsDirectlyToSource: Esta propiedad establece un valor que indica si se va a evaluar la propiedad *Path* con respecto al elemento de datos o al objeto *ObjectDataProvider*. El valor por defecto de esta propiedad es *false*. Cuando es *false* evalúa la ruta de acceso con respecto al elemento de datos propiamente dicho; de lo contrario, cuando es verdadero, evaluará la ruta de acceso respecto al objeto *ObjectDataProvider*. En el ejemplo anterior debe estar en *true*, ya que la propiedad *Path* tiene el valor de una propiedad de *ObjectDataProvider(MethodParameters)* y no del elemento de datos.



DisplayMemberPath: Indica el nombre de la propiedad del objeto de origen que será mostrada en el elemento enlazado.

ItemsSource: Representa la colección que contiene los datos. Se usa cuando se enlaza a colecciones.

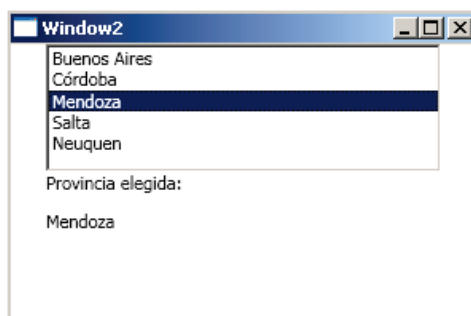
IsSynchronizedWithCurrentItem: Indica si la selección debe mantener la propiedad *SelectedItem* sincronizada con el elemento actual en la propiedad *Items*. Puede establecer el valor de la propiedad *IsSynchronizedWithCurrentItem* en *true* para asegurarse de que el elemento seleccionado siempre se corresponde con la propiedad *CurrentItem* de *ItemCollection*. Se usa mucho en relaciones Cabecera-Detalle. Por ejemplo, suponga que existen dos controles *ListBox* con su propiedad *ItemsSource* establecida en el mismo origen. Establezca la propiedad *IsSynchronizedWithCurrentItem* en *true* en ambos cuadros de lista para asegurarse de que el elemento seleccionado en cada *ListBox* sea el mismo.

Enlace entre Controles

Es posible enlazar dos controles de manera de sincronizar los valores entre ellos.

En el siguiente ejemplo se enlaza la propiedad *Text* de un *TextBlock* a un elemento seleccionado de un control *ListBox*.

```
<StackPanel>
  <ListBox x:Name="lbProvincias" Width="248" Height="80">
    <ListBoxItem Content="Buenos Aires"/>
    <ListBoxItem Content="Córdoba"/>
    <ListBoxItem Content="Mendoza"/>
    <ListBoxItem Content="Salta"/>
    <ListBoxItem Content="Neuquen"/>
  </ListBox>
  <TextBlock Width="248" Height="24" Text="Provincia elegida:" />
  <TextBlock Width="248" Height="24">
    <TextBlock.Text>
      <Binding ElementName="lbProvincias"
        Path="SelectedItem.Content"/>
    </TextBlock.Text>
  </TextBlock>
</StackPanel>
```



La propiedad *Text* del *TextBlock* declara un enlace al elemento seleccionado del *ListBox* con la etiqueta *<Binding>*. El atributo *ElementName* de la etiqueta *Binding* indica el nombre del control al cual está enlazada la propiedad *Text* de *TextBlock*. El atributo *Path* indica la propiedad del elemento (en este caso *ListBox*) a la cual realizaremos el enlace. El resultado de este código es que cuando se selecciona una provincia del *ListBox*, el nombre de esa provincia se muestra en *TextBlock*.



El código anterior también puede escribirse de manera abreviada:

```
<TextBlock Width="248" Height="24"
Text="{Binding ElementName=lbProvincias, Path=SelectedItem.Content}" />
```

XmlDataProvider

El objeto *XmlDataProvider* puede usarse como origen de datos. Este objeto puede enlazar a un documento o fragmento XML que se encuentre incrustado en la etiqueta o en un archivo al que se hace referencia en una ubicación externa.

Se debe dar a *XmlDataProvider* un valor **x:Key** de manera que se pueda hacer referencia a él a través de los destinos de enlace de datos. Tenga en cuenta el atributo **XPath**. Este atributo define el nivel del contenido XML que se usará como el origen de datos. Esto es muy útil cuando se enlaza a una estructura XML grande que puede incluirse en un archivo o base de datos y los datos a los que desea realizar el enlace no constituyen el elemento raíz. El contenido XML incrustado debe colocarse en una etiqueta **<x:XData>**.

En este ejemplo los datos XML están incrustados dentro del XMLDataProvider

```
<StackPanel>
  <StackPanel.Resources>
    <XmlDataProvider x:Key="Colores" XPath="/colores">
      <x:XData>
        <colores>
          <color name="Azul"/>
          <color name="Verde"/>
          <color name="Amarillo"/>
          <color name="Blanco"/>
          <color name="Negro"/>
        </colores>
      </x:XData>
    </XmlDataProvider>
  </StackPanel.Resources>
  <ListBox x:Name="lbColor" Width="248" Height="56"
    ItemsSource="{Binding Source={StaticResource Colores},
    XPath=color/@name}" />
</StackPanel>
```

Se puede crear un archivo XML y luego referenciar dicho archivo usando XMLDataProvider. El ejemplo siguiente enlaza a un archivo llamado Colores.xml.

```
<StackPanel.Resources>
  <XmlDataProvider x:Key="Colores" Source="Colores.xml" XPath="/colores"/>
</StackPanel.Resources>
```

Archivo Colores.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<colores >
  <color name="Azul"/>
  <color name="Verde"/>
  <color name="Amarillo"/>
```



```
<color name="Blanco"/>
<color name="Negro"/>
</colores>
```

Azul
Verde
Amarillo
Blanco
Negro

ObjectDataProvider

Si se desea realizar un enlace a un objeto o a una lista de objetos, se puede utilizar el objeto *ObjectDataProvider* como un recurso. La propiedad **ObjectType** del *ObjectDataProvider* designa el objeto que proporcionará el origen de enlace de datos mientras que la propiedad **MethodName** indica el método que será invocado para obtener los datos.

Existe otra serie de propiedades disponibles en *ObjectDataProvider*. La propiedad **ConstructionParameters** le permite pasar los parámetros al constructor de la clase que se invoca. Puede especificar también los parámetros mediante la propiedad **MethodParameters** y usar la propiedad **ObjectInstance** para especificar una instancia existente de un objeto como el origen.

Si desea que los datos se recuperen de manera asincrónica, puede configurar la propiedad **IsAsynchronous** de *ObjectDataProvider* en *true*. A continuación el usuario podrá interactuar con la pantalla mientras espera que los datos rellenen el control de destino que está enlazado con el origen de *ObjectDataProvider*.

Al agregar un *ObjectDataProvider*, es necesario calificar el espacio de nombres de la clase de origen de datos. En este caso, tengo que agregar un atributo **xmlns** a la etiqueta *<Window>* para que se califique el acceso directo y éste indique el espacio de nombres apropiado.

En el siguiente ejemplo se crea una clase llamada Países la cual contiene un método llamado *TraerPaíses*, el cual devuelve todos los países de una tabla de la base de datos AdventureWorks de SQL Server.

```
public class Países
{
    public DataTable TraerPaíses()
    {
        using (DataTable dt = new DataTable())
        {
            using (SqlConnection cn = new
                SqlConnection(WpfCSharp.Properties.Settings.Default.AdvWorks))
            {
                using (SqlDataAdapter da = new SqlDataAdapter("Select *
                    from Person.CountryRegion", cn))
                {
                    da.Fill(dt);
                }
            }
            return dt;
        }
    }
}
```



Una vez generada la clase podemos generar la referencia al método usando el control *ObjectDataProvider* y asociarla a un control enlazable. Para poder llamar a la clase Países deberá generar la referencia con la línea `xmlns:svc="clr-namespace:WpfCSharp"` al igual que en el uso de controles.

```
<Window x:Class="WpfCSharp.Window2"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  Title="Window2" Height="300" Width="500"
  xmlns:svc="clr-namespace:WpfCSharp">
  <StackPanel>
    <StackPanel.Resources>
      <ObjectDataProvider x:Key="misPaíses" ObjectType="{x:Type
        svc:Países}" MethodName="TraerPaíses"/>
    </StackPanel.Resources>
    <ListBox x:Name="lbPersons" Height="250" Width="200"
      ItemsSource="{Binding Source={StaticResource misPaíses}}"
      DisplayMemberPath="Name" />
  </StackPanel>
</Window>
```



MethodParameters

Permite armar la lista de parámetros para pasarlos en la llamada al método definido en la propiedad *MethodName*.

El ejemplo siguiente muestra una clase Provincias que contiene un método TraerProvincias que recibe un parámetro de tipo carácter que permite devolver solo las provincias asociadas al código del país recibido.

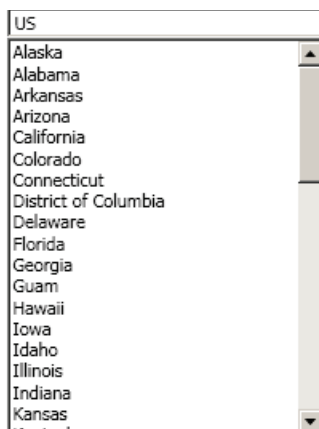
```
public class Provincias
{
    public DataTable TraerProvincias(string Cod)
    {
        using (DataTable dt = new DataTable())
        {
            using (SqlConnection cn = new
                SqlConnection(WpfCSharp.Properties.Settings.Default.AdvWorks))
```



```
{
    using (SqlDataAdapter da = new SqlDataAdapter("Select *
    From Person.stateProvince Where
    CountryRegionCode=@Cod", cn))
    {
        da.SelectCommand.Parameters.Add(new
        SqlParameter("@Cod", SqlDbType.Char, 3));
        da.SelectCommand.Parameters["@Cod"].Value = Cod;
        da.Fill(dt);
    }
}
return dt;
}
}
```

Usando *ObjectDataProvider* generamos la referencia a este método y con *MethodParameters* definimos su parámetro. Luego asociamos ese parámetro a un *TextBox* de manera de poder seleccionar distintos valores y mostramos la lista de provincias en un *ListBox*.

```
<Window x:Class="WpfCSharp.Window2"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Window2" Height="300" Width="500"
    xmlns:svc="clr-namespace:WpfCSharp"
    xmlns:system="clr-namespace:System;assembly=microsoft.dll" >
    <StackPanel>
        <StackPanel.Resources>
            <ObjectDataProvider x:Key="ProvXPais" ObjectType="{x:Type svc:Provincias}"
                MethodName="TraerProvincias">
                <ObjectDataProvider.MethodParameters>
                    <system:String>US</system:String>
                </ObjectDataProvider.MethodParameters>
            </ObjectDataProvider>
        </StackPanel.Resources>
        <TextBox Name="txtPais" Width="200">
            <TextBox.Text>
                <Binding Source="{StaticResource ProvXPais}"
                    Path="MethodParameters[0]"
                    BindsDirectlyToSource="true"
                    UpdateSourceTrigger="PropertyChanged"/>
            </TextBox.Text>
        </TextBox>
        <ListBox x:Name="lbPersons" Height="250" Width="200"
            ItemsSource="{Binding Source={StaticResource ProvXPais}}"
            DisplayMemberPath="Name" />
    </StackPanel>
</Window>
```



DataTemplate

Se usa para generar una visualización de los datos personalizada. DataTemplate es el objeto que permite armar la plantilla de datos.

En el siguiente ejemplo usa DataTemplate para modificar la apariencia de la lista de manera que el nombre del país se vea en negrita.

```
<Window x:Class="WpfCSharp.Window2"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  Title="Window2" Height="300" Width="500"
  xmlns:svc="clr-namespace:WpfCSharp">
  <StackPanel>
    <StackPanel.Resources>
      <ObjectDataProvider x:Key="misPaises" ObjectType="{x:Type svc:Paises}"
        MethodName="TraerPaises"/>
      <DataTemplate x:Key="LayoutPais" >
        <StackPanel Orientation="Horizontal" >
          <TextBlock Text="{Binding Path=CountryRegionCode}" />
          <TextBlock Text=" - " />
          <TextBlock Text="{Binding Path=Name}"
            FontWeight="Bold" />
        </StackPanel>
      </DataTemplate>
    </StackPanel.Resources>
    <ListBox x:Name="lbPersons" Height="250" Width="200"
      ItemsSource="{Binding Source={StaticResource misPaises}}"
      ItemTemplate="{DynamicResource LayoutPais}" />
  </StackPanel>
</Window>
```



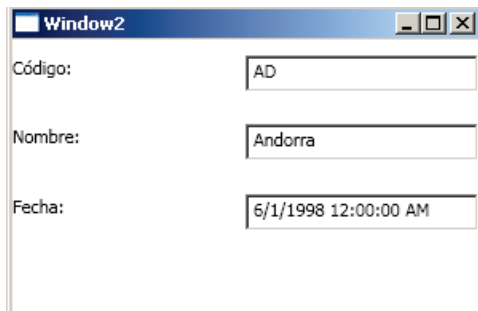
DataContext

Todos los controles que derivan de *FrameworkElement* tienen la propiedad *DataContext*. Esta propiedad permite relacionarse con los datos. La propiedad *DataContext* es heredada por los elementos hijos del elemento al cual está asociada, por lo cual se recomienda asociarla a los elementos contenedores. Es muy útil para construir formularios en donde varias propiedades se asocian a datos de la misma fuente de datos.

En el siguiente ejemplo enlaza el origen de datos al objeto Grid, de esta manera que todos los controles contenidos especifican solo el nombre del campo ya que el enlace principal está asociado a su contenedor.

```
<Window x:Class="WpfCSharp.Window2"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  Title="Window2" Height="300" Width="500">
  <Window.Resources>
    <ObjectDataProvider x:Key="misPaíses" ObjectType="{x:Type
      svc:Países}" MethodName="TraerPaíses"/>
  </Window.Resources>
  <Grid DataContext="{StaticResource misPaíses}">
    <Grid.ColumnDefinitions>
      <ColumnDefinition />
      <ColumnDefinition />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
      <RowDefinition />
      <RowDefinition />
      <RowDefinition />
      <RowDefinition />
    </Grid.RowDefinitions>
    <TextBlock Grid.Column="0" Grid.Row="0" Height="21" Name="txbCodigo"
      Text="Código:" />
    <TextBlock Grid.Column="0" Grid.Row="1" Height="21" Name="txbNombre"
      Text="Nombre:" />
    <TextBlock Grid.Column="0" Grid.Row="2" Height="21" Name="txbFecha"
      Text="Fecha:" />
    <TextBox Grid.Column="1" Grid.Row="0" Height="23" Name="txtCodigo"
      Text="{Binding CountryRegionCode}"/>
  </Grid>
</Window>
```

```
<TextBox Grid.Column="1" Grid.Row="1" Height="23" Name="txtNombre"
Text="{Binding Name}"/>
<TextBox Grid.Column="1" Grid.Row="2" Height="23" Name="txtFecha" Text="{Binding
ModifiedDate}"/>
</Grid>
</Window>
```



Controles ListView - Gridview

WPF no tiene un control *GridView* pero este puede generarse usando un control *ListView*. *ListView* contiene una propiedad *View* que provee un objeto *GridView*. La propiedad *Columns* de un *GridView* contiene elementos del tipo *GridViewColumn*. La propiedad *AllowsColumnReorder* permite establecer si la columnas soportan ordenado o no.

En el siguiente ejemplo usando la misma clase Países declarada en el ejemplo anterior generamos un *ListView* y organizamos las columnas del mismo usando el objeto *GridView*.

```
<Window.Resources>
    <ObjectDataProvider x:Key="misPaíses" ObjectType="{x:Type svc:Países}"
        MethodName="TraerPaíses"/>
</Window.Resources>
<ListView Height="150" Width="280" ItemsSource="{Binding Source={StaticResource
misPaíses}}">
    <ListView.View>
        <GridView>
            <GridView.Columns>
                <GridViewColumn Width="50" Header="Código"
                    DisplayMemberBinding="{Binding
                    Path=CountryRegionCode}"/>
                <GridViewColumn Width="200" Header="Nombre"
                    DisplayMemberBinding="{Binding Path=Name}"/>
            </GridView.Columns>
        </GridView>
    </ListView.View>
</ListView>
```

Código	Nombre
AD	Andorra
AE	United Arab Emirates
AF	Afghanistan
AG	Antigua and Barbuda
AI	Anguilla
AL	Albania
AM	Armenia
AN	Netherlands Antilles
AO	Angola
AS	American Samoa